

Added Playwright/JavaScript language combination in Fine tuning test data builder.

Prompt –

Context

Act as a senior software developer and analyse the fine-tuning-test-data-builder.html and add new framework combination – Playwright/JavaScript.

Instructions

[Mandatory] Analyse the fine-tuning-test-data-builder.html

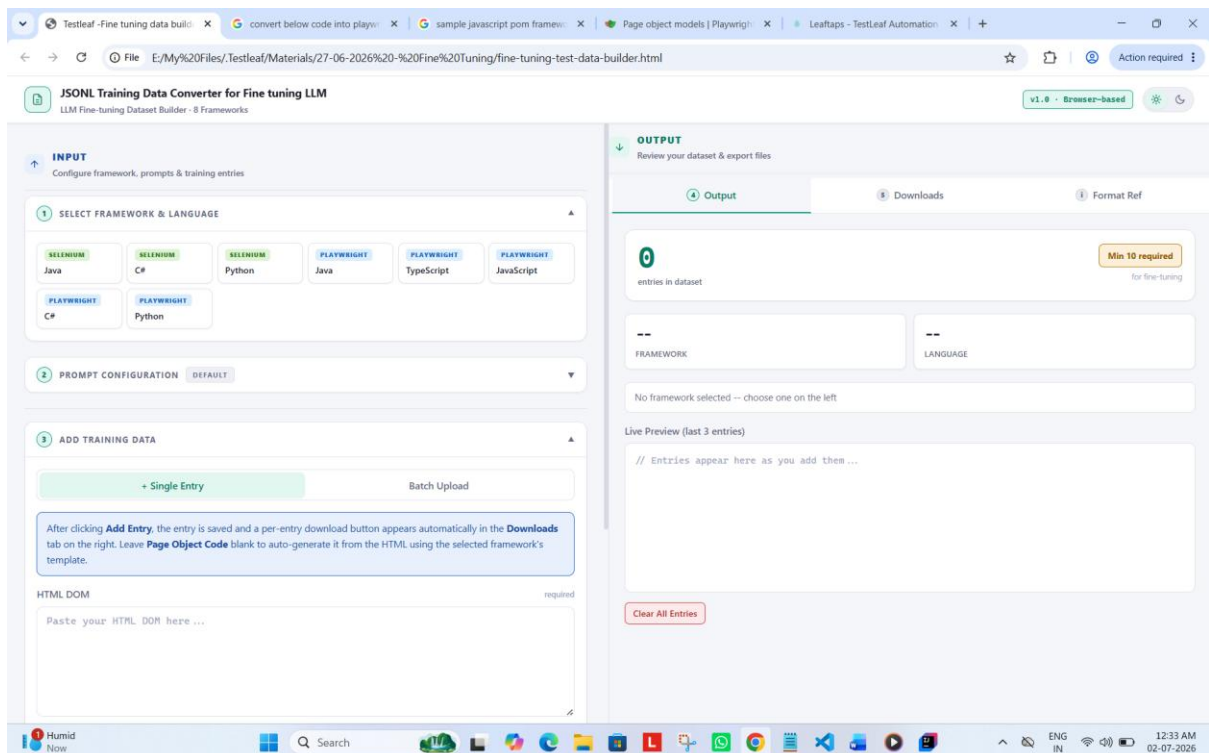
[Mandatory] Add new framework combination – Playwright/JavaScript

[STRICT] Should not change existing implementation

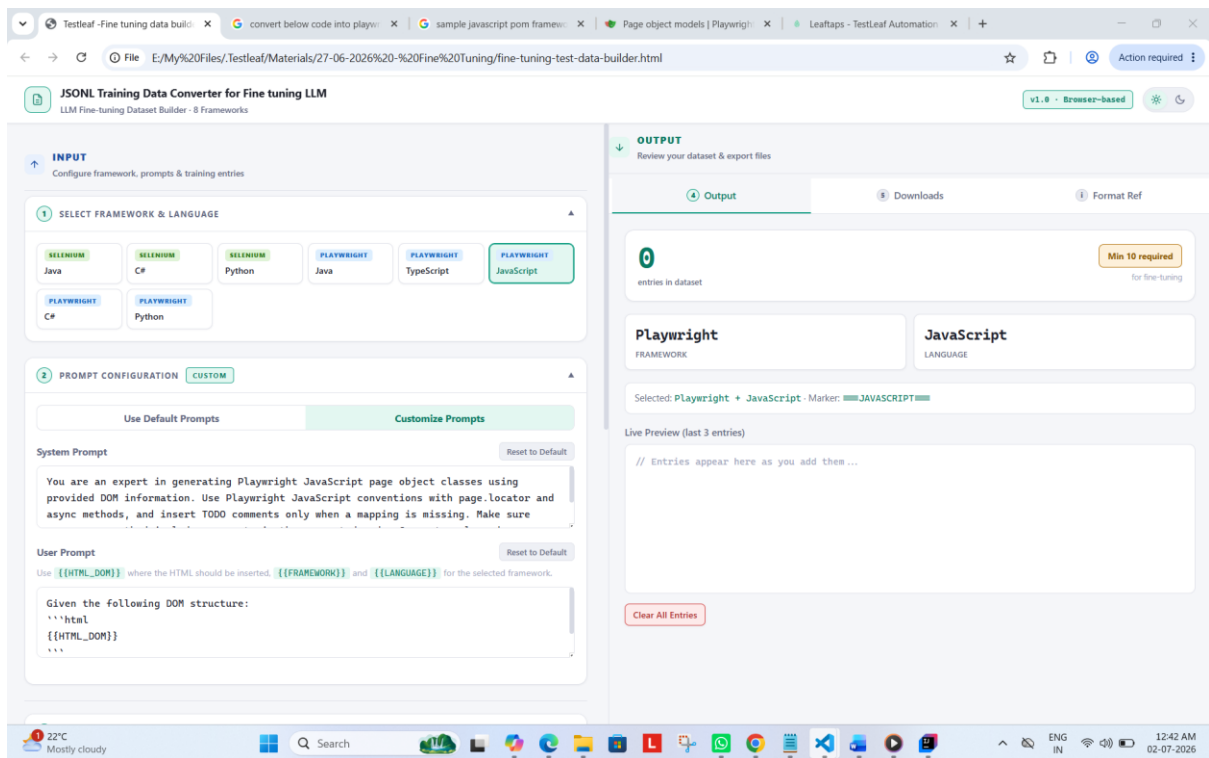
[Mandatory] Should be able to generate jsonl file

Output

Give the updated html page as output. Which supports Playwright/JavaScript.

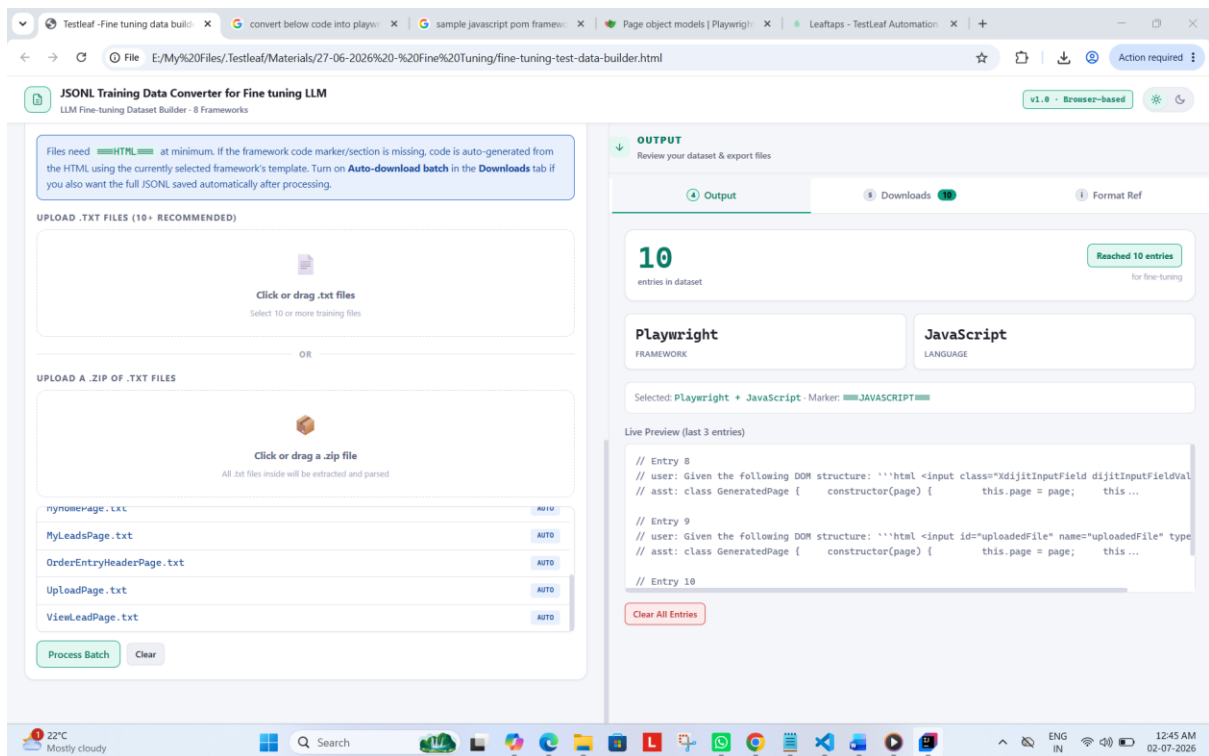


Customized prompt – based on selection



The screenshot shows the 'JSONL Training Data Converter for Fine tuning LLM' interface. The 'INPUT' section is active, showing 'SELECT FRAMEWORK & LANGUAGE' with 'PLAYWRIGHT' and 'JAVASCRIPT' selected. The 'PROMPT CONFIGURATION' section is also visible, showing 'CUSTOM' prompts. The 'OUTPUT' section shows '0 entries in dataset' and a 'Min 10 required' warning. The 'Live Preview' shows the generated JSONL entries for the selected framework and language.

Processed Batch



The screenshot shows the 'JSONL Training Data Converter for Fine tuning LLM' interface. The 'UPLOAD .TXT FILES' section is active, showing a list of files: 'nynomerPage.txt', 'MyLeadsPage.txt', 'OrderEntryHeaderPage.txt', 'UploadPage.txt', and 'ViewLeadPage.txt'. The 'OUTPUT' section shows '10 entries in dataset' and a 'Reached 10 entries' status. The 'Live Preview' shows the generated JSONL entries for the selected framework and language.

Jsonl – downloads

```
{
  "messages": [
    {
      "role": "system",
      "content": "You are an expert in generating Playwright JavaScript page object classes using provided DOM information. Use Playwright JavaScript conventions with page.locator and async methods, and insert TODO comments only when a mapping is missing. Make sure every page method includes comments in the generated code. Generate only code grounded in the supplied DOM."
    },
    {
      "role": "user",
      "content": "Given the following DOM structure:\n`html\n<input type='text' class='inputBox' name='companyName' size='30' maxlength='100' id='createLeadForm_companyName'>\n<input type='text' class='inputBox' name='firstName' size='30' maxlength='100' id='createLeadForm_firstName'>\n<input type='text' class='inputBox' name='lastName' size='30' maxlength='100' id='createLeadForm_lastName'>\n<input type='submit' class='smallSubmit' name='submitButton' value='Create Lead' id='ext-gen652'>\n\n====JavaScript====\n// editLeadPage.js\n\nclass EditLeadPage {\n  /**\n   * @param {import('@playwright/test').Page} page\n   *\n   * constructor(page)\n   *\n   * this.page = page;\n   *\n   * // Locators\n   * this.companyNameInput = page.locator('#createLeadForm_companyName');\n   *\n   * this.firstNameInput = page.locator('#createLeadForm_firstName');\n   *\n   * this.lastNameInput = page.locator('#createLeadForm_lastName');\n   *\n   * this.createLeadButton = page.locator('//input[@value='create lead']');\n   *\n   *}\n   *\n   * async enterCompanyName(companyName) {\n   *   await this.companyNameInput.fill(companyName);\n   *   return this;\n   * }\n   *\n   * async enterFirstName(firstName) {\n   *   await this.firstNameInput.fill(firstName);\n   *   return this;\n   * }\n   *\n   * async enterLastName(lastName) {\n   *   await this.lastNameInput.fill(lastName);\n   *   return this;\n   * }\n   *\n   * async clickCreateLead() {\n   *   await this.createLeadButton.click();\n   *   return this;\n   * }\n   *\n   *}\n\nmodule.exports = { EditLeadPage };"`
    },
    {
      "role": "assistant",
      "content": "class CreateLeadPage {\n  constructor(page) {\n    this.page = page;\n    this.companyNameInput = null;\n    this.firstNameInput = null;\n    this.lastNameInput = null;\n    this.submitButton = null;\n\n    this.companyNameInput = page.locator('#createLeadForm_companyName');\n    this.firstNameInput = page.locator('#createLeadForm_firstName');\n    this.lastNameInput = page.locator('#createLeadForm_lastName');\n    this.submitButton = page.locator('#ext-gen652');\n\n    /** Fill the companyName field */\n    async enterCompanyName(companyName) {\n      await this.companyNameInput.fill(companyName);\n    }\n  }\n}"
```

A screenshot of a Windows 10 desktop environment. The primary focus is a code editor window titled 'formatted (1).json' located at 'Downloads > formatted (1).json'. The editor displays a JSON file with two main properties: 'messages' and 'role'. The 'messages' property contains a 'content' field with a large, multi-line JavaScript code block. This code appears to be a Playwright test script for a 'Create Lead' form, including imports, page object definitions, and test steps like 'enterCompanyName', 'enterFirstName', 'enterLastName', and 'clickSubmitButton'. The 'role' property also contains a 'content' field with a smaller JavaScript code block defining a 'GeneratedPage' class. The code is syntax-highlighted, with strings in quotes, comments in /* */, and keywords in blue. To the right of the code editor, a vertical sidebar shows a file explorer view with a tree of folders and files. The Windows taskbar is visible at the bottom, featuring the Start button, a search bar, and several pinned application icons including File Explorer, Edge, and various utility tools. The system tray on the far right shows the date and time as '12:50 AM 02-07-2020'.